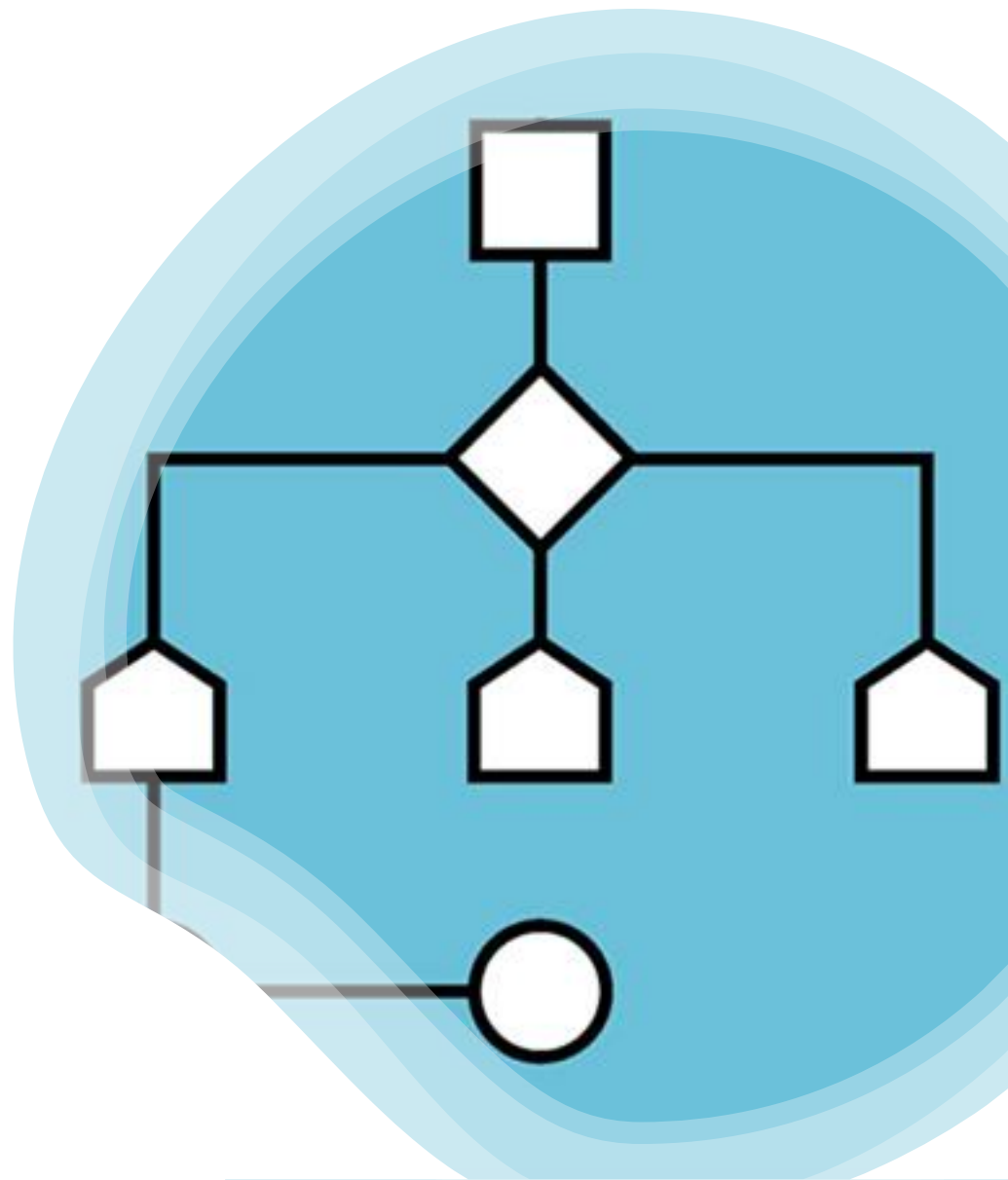
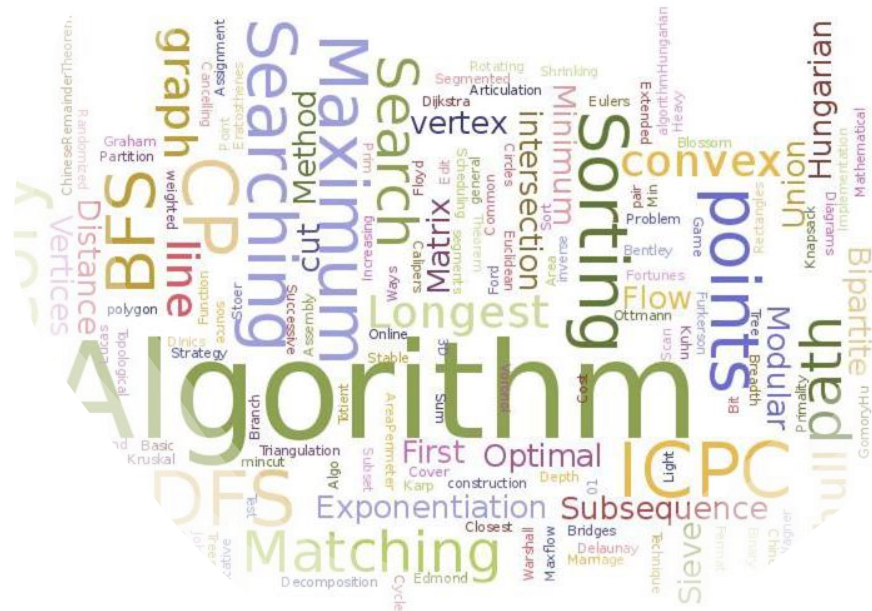


# LA RECHERCHE DICHOTOMIQUE



# Déroulement du TP

- Principe des algorithmes diviser pour régner
- Principe de l'algorithme de recherche dichotomique
- Implémentation de la recherche dichotomique en version "itérative" :
  - Prototype de la fonction de recherche dichotomique
  - Définition de la fonction de recherche dichotomique
- Création d'un jeu de test pour valider le code imaginé.

# Principe des algorithmes



---

**Diviser pour Régner**

# DIVISER POUR RÉGNER

---



Le principe est que si un problème est complexe à résoudre, alors on va le segmenter (découper) en 2 ou plusieurs parties.



Chaque partie est alors plus simple à résoudre. La résolution des différents segments peut se faire séquentiellement ou parallèlement selon l'algorithme et les capacités de la machine.



Une fois que tous les segments sont traités, on rassemble le tout.

# DIVISER POUR RÉGNER

- Un certain nombre d'algorithmes se prêtent bien à la stratégie du diviser pour régner :
  - Recherche dichotomique (objet de la suite de ce TP)
  - Tri fusion
  - Tri rapide
  - Etc.

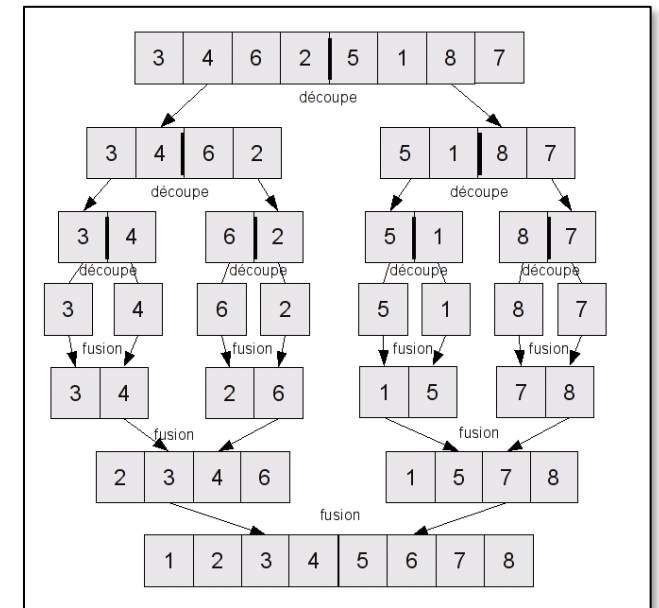
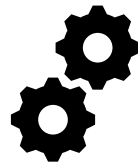


Illustration du tri fusion

# L'algorithme de recherche dichotomique

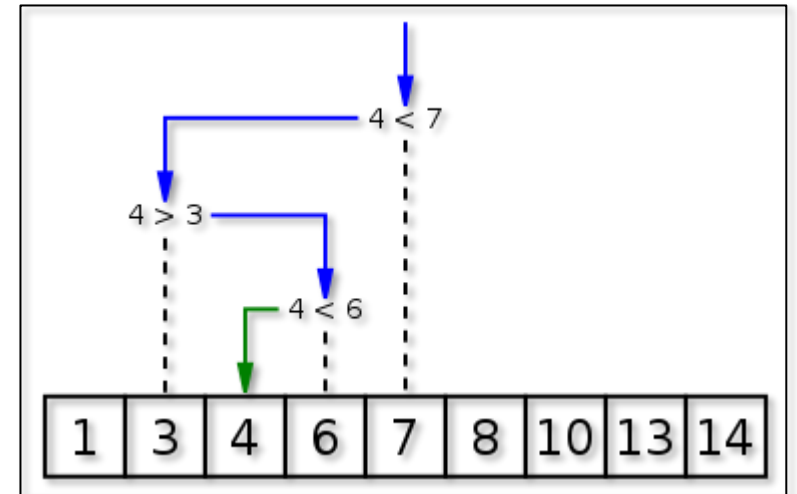


---

**Principe**

# RECHERCHE DICHOTOMIQUE

- **Condition :** L'algorithme ne fonctionne que sur une liste **triée** !
- **Description :**
  - **Comparer** la valeur à trouver avec la valeur située **au milieu** du tableau;
  - Si la valeur recherchée est égale à la valeur centrale, la recherche est terminée !
  - Dans le cas contraire, prendre la moitié du tableau pouvant contenir la valeur et réitérer l'algorithme jusqu'à trouver la valeur recherchée.



Principe pour la recherche de la valeur 4

# Implémentation de l'algorithme



---

**Version itérative**



# IMPLÉMENTATION VERSION ITÉRATIVE

- Créer un nouveau fichier Python
- Rédiger le prototype de la fonction, c'est-à-dire rédiger l'entête de la fonction, sa documentation et vous préciserez la valeur retournée par la fonction.
- Vous présenterez le prototype de votre fonction à votre professeur avant de poursuivre.
  - Pour exemple : Prototype d'une fonction qui retourne le nombre d'occurrences d'une valeur contenue dans une liste :

```
1 def nbrOccurrenceListe(val, liste):
2     """Fonction retournant le nombre d'occurrences de la valeur val
3     donnée en argument. Le 2e argument est la liste dans laquelle est
4     menée la recherche."""
5
6     nbrOcc = 0 # Nombre d'occurrence de la valeur recherchée.
7
8     # DEFINITION DE LA FONCTION
9
10    return nbrOcc
```

**Entête**

# IMPLÉMENTATION VERSION ITÉRATIVE

- Après validation par votre professeur, réaliser la définition de votre fonction de recherche par dichotomie.
- Procéder à un test avec une liste et une valeur de votre choix contenue dans votre liste et une valeur non contenue dans la liste.

```
>>> %Run recherche_dichotomique.py  
Liste d'essai : [-3, 1, 3, 5, 9, 10, 13, 14]  
Résultat de recherche de la valeur 13 : 6  
Résultat de recherche de la valeur 7 : None
```

# Réalisation d'un jeu de test

**Validation du code par une série de tests**

# IMPLÉMENTATION VERSION ITÉRATIVE

- Imaginer un jeu de test permettant de valider le bon déroulement de votre fonction.
- Pour être efficace, nous vous conseillons d'élaborer un jeu de test qui va réaliser sur une liste, l'exécution de votre fonction **dicho()** pour chacune de ses valeurs.
- **Exemple de jeu de test :**
  - Générer une liste de taille T avec des valeurs classées dans l'ordre croissant.
  - Réaliser une boucle qui lance un appel de la fonction **dicho()** pour chaque valeur de la liste.

```
>>> %Run dicho.py
Liste scrutée : [0, 1, 2, 3, 4]
Valeur à chercher : 0 est à l'indice : 0
Valeur à chercher : 1 est à l'indice : 1
Valeur à chercher : 2 est à l'indice : 2
Valeur à chercher : 3 est à l'indice : 3
Valeur à chercher : 4 est à l'indice : 4

Cas où la valeur n'est pas présente :
La valeur -5 est dans liste ? : None
```

FIN